

FuelMate

Aplikacja wspierająca kierowców w planowaniu tankowania oraz
ostrzegająca przed zagrożeniami drogowymi

Projektowanie Aplikacji Internetowych **Projekt nr 4**

Dokumentacja etapów 1–3
11.05.2026

Autorzy: Igor Stduziński, Kacper Chlastawa, Aneta Młynarczyk, Benedykt Okarmus

Spis treści

1. Określenie zagadnienia biznesowego	3
1.1. Opis problemu	3
1.2. Grupa docelowa	3
1.3. Proponowane rozwiązanie	3
1.4. Model biznesowy	4
2. Określenie wymagań funkcjonalnych	5
2.1. Aktorzy systemu	5
2.2. Wymagania funkcjonalne	5
2.3. Wymagania niefunkcjonalne	7
2.4. Widok profilu użytkownika	7
3. Projektowanie architektury aplikacji	8
3.1. Założenia architektoniczne	8
3.2. Diagram architektury	8
3.3. Szczegółowy opis serwisów	9
3.4. Komunikacja międzyserwisowa	10
3.5. Warstwa prezentacji	11
3.6. Bazy danych	12
3.7. Bezpieczeństwo i konfiguracja	12

1. Określenie zagadnienia biznesowego

Etap 1 – Identyfikacja problemu i propozycja rozwiązania

1.1. Opis problemu

Współcześni kierowcy w Polsce stoją przed wyzwaniem efektywnego zarządzania kosztami eksploatacji pojazdu. Ceny paliw różnią się znacząco między stacjami, a dostęp do aktualnych danych jest ograniczony. Istniejące rozwiązania (np. Yanosik) koncentrują się głównie na alertach drogowych, pomijając aspekt planowania tankowania i transparentności cen paliw. Dodatkowo, brakuje mechanizmów gamifikacji motywujących użytkowników do aktywnego raportowania cen i zagrożeń.

1.2. Grupa docelowa

- **Kierowcy prywatni** poszukujący najtańszych stacji paliw w okolicy.
- **Kierowcy zawodowi** (taksówkarze, kurierzy) monitorujący koszty paliwa.
- **Floty samochodowe** potrzebujące analityki cenowej dla optymalizacji tras.
- **Użytkownicy aplikacji społecznościowych** chętni do dzielenia się informacjami drogowymi.

1.3. Proponowane rozwiązanie

Aplikacja **FuelMate** stanowi platformę łączącą trzy kluczowe funkcjonalności:

1. **Mapę stacji paliw** z aktualnymi cenami raportowanymi przez społeczność.
2. **System alertów drogowych** (patrol, fotoradar, kontrola, wypadek).
3. **Mechanizm gamifikacji** oparty na punktach za aktywność użytkowników.

Dzięki wykorzystaniu geolokalizacji oraz architektury mikroserwisowej, system zapewnia skalowalność, niezawodność i łatwość rozbudowy.

1.4. Model biznesowy i uzasadnienie

Rozwiązanie realizuje następujące cele biznesowe:

1. **Redukcja kosztów tankowania** – użytkownik widzi najkorzystniejsze ceny w swojej okolicy.
2. **Zwiększenie bezpieczeństwa drogowego** – bieżące alerty o patrolach i zagrożeniach.
3. **Budowanie społeczności** – system punktowy motywuje do aktywnego uczestnictwa.
4. **Analityka dla biznesu** – stacje paliw mogą analizować ceny konkurencji (funkcja rozwojowa).

Kluczowe przewagi konkurencyjne: transparentcja cen w czasie rzeczywistym, połączenie alertów drogowych z lokalizacją stacji, system rankingowy budujący zaangażowanie oraz architektura mikroserwisowa umożliwiająca niezależny rozwój modułów.

2. Określenie wymagań funkcjonalnych

Etap 2 – Identyfikacja funkcjonalności systemu

2.1. Aktorzy systemu

Aktor	Opis
Gość (Guest)	Użytkownik niezalogowany; może przeglądać mapę, ceny paliw i alerty drogowe w swojej okolicy.
Użytkownik	Zalogowany kierowca; raportuje ceny, dodaje alerty, ocenia stacje, zbiera punkty i rywalizuje w rankingu.
Administrator	Uprawniony użytkownik (flaga isAdmin); zarządza kontami, moderuje treści i ma dostęp do statystyk.

2.2. Wymagania funkcjonalne według modułów

Moduł Autentykacji i Użytkowników (Auth Service)

F-AU-01	Rejestracja konta z walidacją danych (email unikalny, hasło minimum 6 znaków).
F-AU-02	Logowanie z wykorzystaniem JWT (JSON Web Token) z czasem życia 24h.
F-AU-03	Przechowywanie profilu użytkownika (imię, nazwisko, miasto, kod pocztowy, punkty).
F-AU-04	Ranking użytkowników generowany na podstawie zebranych punktów (Top 10).
F-AU-05	Rozróżnienie ról: użytkownik standardowy vs administrator (isAdmin).
F-AU-06	Widok profilu użytkownika z danymi osobowymi, punktami, historią aktywności i ulubionymi stacjami.

Moduł Stacji i Cen Paliw (Station Service)

F-ST-01	Wyświetlanie stacji paliw na mapie z aktualnymi cenami (geolokalizacja GPS).
F-ST-02	Filtrowanie stacji w promieniu określonym przez użytkownika (formuła haversine).
F-ST-03	Dodawanie nowej ceny paliwa dla wybranego typu (PB95, PB98, ON, LPG, EV).
F-ST-04	Automatyczne przyznawanie punktów użytkownikowi za raportowanie cen (+1 pkt).
F-ST-05	Ocenianie stacji w skali 1–5 gwiazdek z opcjonalnym komentarzem.
F-ST-06	Przeliczanie średniej oceny i liczby głosów dla każdej stacji.

Moduł Alertów Drogowych (Alert Service)

F-AL-01	Zgłaszanie alertów typu: patrol, fotoradar, wypadek, kontrola, kamera.
F-AL-02	Automatyczna lokalizacja GPS przy dodawaniu alertu.
F-AL-03	Wygasanie alertów po 3 godzinach od zgłoszenia (mechanizm TTL).
F-AL-04	Potwierdzanie istnienia alertu przez innych użytkowników (unikalność na użytkownika).
F-AL-05	Przyznawanie punktów za zgłoszenie alertu (+2 pkt) i jego potwierdzenie.

Moduł Ulubionych i Powiadomień (Favorite Service)

F-FA-01	Dodawanie stacji do listy ulubionych dla zalogowanego użytkownika.
F-FA-02	Usuwanie stacji z listy ulubionych.
F-FA-03	Przeglądanie listy ulubionych z podstawowymi informacjami o stacjach.
F-FA-04	Mechanizm powiadomień o zmianie ceny w ulubionej stacji (funkcja rozwojowa).

2.3. Wymagania niefunkcjonalne

Kategoria	Opis wymagania
Bezpieczeństwo	Hasła hashowane bcrypt (10 rund), komunikacja przez JWT, walidacja danych wejściowych.
Wydajność	Asynchroniczna komunikacja klient-serwer (Axios + REST), indeksy geograficzne w bazie.
Skalowalność	Architektura mikroservisowa umożliwia niezależne skalowanie poszczególnych modułów.
Dostępność	CSR (Client Side Rendering) zapewnia płynne działanie interfejsu bez przeładowań.
Konfigurowalność	Adresy serwisów, porty, sekrety kryptograficzne i ścieżki baz danych w plikach <code>.env</code> .
Responsywność	Interfejs dostosowany do urządzeń mobilnych i desktopów.

2.4. Widok profilu użytkownika

Widok profilu stanowi centralny punkt zarządzania tożsamością użytkownika w aplikacji. Dostępny po kliknięciu nazwy użytkownika w pasku górnym (zamiast prostego wylogowania). Zawiera następujące sekcje:

- **Dane osobowe** – imię, nazwisko, email, miasto, kod pocztowy (możliwość edycji).
- **Statystyki** – aktualna liczba punktów, pozycja w rankingu, liczba zgłoszonych cen i alertów.
- **Ulubione stacje** – lista zapisanych stacji z aktualnymi cenami (pobierana z Favorite Service).
- **Historia aktywności** – ostatnie zgłoszenia cen i alertów (w przeszłości).
- **Zarządzanie kontem** – zmiana hasła, wylogowanie.

Komponent `UserProfile.jsx` komunikuje się z Auth Service (dane profilu), Favorite Service (ulubione) oraz Station Service (ceny w ulubionych). Profil wymaga autentykacji JWT.

3. Projektowanie architektury aplikacji

Etap 3 – Architektura mikroserwisowa i podział funkcjonalny

3.1. Założenia architektoniczne

Aplikacja **FuelMate** zaprojektowana została w oparciu o architekturę mikroserwisową, co pozwala na niezależny rozwój, wdrażanie i skalowanie poszczególnych komponentów. Każdy serwis posiada własne API REST, własną bazę danych SQLite oraz dedykowaną domenę biznesową. Komunikacja między serwisami odbywa się asynchronicznie poprzez protokół HTTP/REST.

Frontend jako aplikacja React (CSR) komunikuje się wyłącznie z **API Gateway**, który pełni rolę jednego punktu wejścia (Single Entry Point) do całego ekosystemu backendowego.

3.2. Szczegółowy opis serwisów

API Gateway port 3000

Technologie: Express.js, cors, dotenv.

Jedyny punkt wejścia dla frontendu. Odpowiedzialny za routing żądań do właściwych mikroserwisów, obsługę CORS, ewentualne rate-limiting oraz agregację odpowiedzi. Eliminuje konieczność bezpośredniej komunikacji klienta z wieloma serwisami backendowymi.

Auth Service port 3001

Technologie: Express, bcryptjs, jsonwebtoken, sequelize, sqlite3.

Dedykowany serwis autentykacji. Zarządza rejestracją, logowaniem, generowaniem i walidacją JWT. Przechowuje dane użytkowników (profil, punkty, rola). Udostępnia endpoint rankingowy (Top 10). Posiada własną bazę SQLite z tabelą users.

Station Service port 3002

Technologie: Express, sequelize, sqlite3.

Serwis domeny stacji paliw. Zarządza encjami: Station, FuelType, Price, StationRating. Implementuje obliczanie odległości haversine dla filtracji geograficznej. Obsługuje dodawanie cen, ocenianie stacji oraz aktualizację średniej oceny. Posiada własną bazę SQLite.

Alert Service port 3003

Technologie: Express, sequelize, sqlite3.

Serwis alertów drogowych. Zarządza encjami: Alert, AlertConfirmation. Implementuje logikę wygasania alertów (TTL 3h), potwierdzeń przez użytkowników oraz filtrację alertów w promieniu od pozycji GPS. Posiada własną bazę SQLite.

Favorite Service port 3004

Technologie: Express, sequelize, sqlite3.

Serwis ulubionych stacji i powiadomień. Zarządza encją Favorite oraz (w przyszłości) subskrypcjami na zmiany cen. Umożliwia zapisywanie, usuwanie i przeglądanie ulubionych stacji dla zalogowanego użytkownika. Posiada własną bazę SQLite.

3.4. Komunikacja międzyserwisowa i przepływ danych

Komunikacja w systemie FuelMate opiera się wyłącznie na asynchronicznych żądaniach HTTP/REST. Frontend nie komunikuje się bezpośrednio z żadnym z mikroserwisów – wszystkie żądania kierowane są do API Gateway, który na podstawie ścieżki routuje je do odpowiedniego serwisu.

Przykładowy przepływ – dodanie ceny paliwa:

1. Użytkownik wybiera stację i wpisuje cenę w interfejsie React.
2. Frontend wysyła POST /api/stations/42/prices do API Gateway.
3. Gateway weryfikuje token JWT (lub przekazuje do Auth Service do walidacji).

4. Gateway przekazuje żądanie do Station Service (port 3002).
5. Station Service zapisuje cenę w swojej bazie SQLite.
6. Station Service wywołuje Auth Service (PATCH /users/:id/points) aby dodać +1 punkt.
7. Odpowiedź propaguje się z powrotem do użytkownika.

Mapowanie endpointów na serwisy

Endpoint (Gateway)	Serwis docelowy	Opis
/api/auth/*	Auth Service (3001)	Rejestracja, logowanie, JWT
/api/ranking	Auth Service (3001)	Top 10 użytkowników
/api/stations/*	Station Service (3002)	Stacje, ceny, oceny
/api/alerts/*	Alert Service (3003)	Alerty, potwierdzenia
/api/favorites/*	Favorite Service (3004)	Ulubione, powiadomienia

3.5. Warstwa prezentacji (Frontend)

Frontend zrealizowany został jako **Single Page Application (SPA)** w technologii React 18 z wykorzystaniem biblioteki Vite jako bundlera i narzędzia deweloperskiego. Renderowanie odbywa się po stronie klienta (CSR). Do wizualizacji map wykorzystano bibliotekę Leaflet wraz z react-leaflet.

Stan autentykacji zarządzany jest przez **React Context** (AuthContext) z persystencją w `localStorage`. Komunikacja z backendem realizowana jest asynchronicznie przez axios z automatycznym dołączaniem nagłówka `Authorization: Bearer <token>`.

Struktura komponentów frontendu

- **App.jsx** – główny komponent, dostarcza AuthProvider i BrowserRouter.
- **MapView.jsx** – główny widok mapy z markerami stacji, alertów i pozycji GPS.
- **StationPanel.jsx** – panel boczny ze szczegółami stacji, cenami, formularzami.
- **AuthModal.jsx** – modal logowania / rejestracji z walidacją.
- **AlertModal.jsx** – modal zgłaszania alertów z wyborem typu i lokalizacją GPS.
- **UserProfile.jsx** – widok profilu użytkownika z danymi, statystykami, ulubionymi i historią aktywności.

3.6. Warstwa danych – bazy danych

Każdy serwis posiada własną, niezależną bazę danych SQLite, co zapewnia separację danych i autonomię poszczególnych modułów. Wzorzec ten odpowiada założeniom architektury mikroserwisowej, gdzie każdy serwis jest właścicielem swojej domeny danych. ORM Sequelize zapewnia abstrakcję nad bazą, migracje schematu oraz definicje relacji.

Przydział tabel do serwisów

Serwis	Tabele
Auth Service	users
Station Service	stations, fuel_types, prices, station_ratings
Alert Service	alerts, alert_confirmations
Favorite Service	favorites

3.7. Bezpieczeństwo i konfiguracja

Wszystkie wrażliwe parametry konfiguracyjne (porty, adresy serwisów, sekrety JWT, ścieżki baz danych) przechowywane są w plikach `.env` poszczególnych serwisów, a nie wpisane na sztywno w kodzie źródłowym.

Mechanizm autentykacji oparty jest o **JSON Web Token (JWT)** z podpisem HMAC-SHA256. Token przesyłany jest w nagłówku `Authorization: Bearer <token>` i walidowany przez middleware `authenticateToken` w każdym chronionym endpointzie. Hasła użytkowników hashowane są algorytmem **bcrypt** z kosztem 10 rund.